TP 2

Process mining avec PM4Py

Contexte

Une entreprise a modélisé son processus de **demande de congé** en BPMN : l'employé dépose sa demande, le manager la valide (ou la refuse), puis le congé est enregistré, le planning mis à jour et l'employé notifié. La direction veut savoir si ce processus est **réellement suivi**, et pourquoi certaines demandes prennent beaucoup de temps. Le système d'information a enregistré **un journal d'événements** de 500 demandes : à vous de l'analyser.

Objectifs

- charger un journal d'événements et en extraire les statistiques de base (cas, activités, variantes) ;
- **découvrir** automatiquement le modèle du processus réel ;
- vérifier la **conformité** du journal par rapport au modèle de référence et trouver les écarts ;
- analyser la **performance** : durées, temps d'attente, goulots.

Fichiers fournis

Fichier	Contenu
`demande_conge.csv`	le journal d'événements : une ligne par événement, avec les colonnes `case_id` (numéro de la demande), `activity` (activité), `timestamp` (date et heure) et `resource` (personne)
`modele_reference_conge.bpmn`	le modèle BPMN prévu par l'entreprise (modèle de référence)

Partie A : Charger et explorer le journal

A.1 — Préparer l'environnement

1. Ouvrez `https://colab.research.google.com` (compte Google), puis **Nouveau notebook**.
   → *Colab exécute Python dans le navigateur : rien à installer sur votre PC.*
2. Dans la première cellule, exécutez la commande ci-dessous (Maj + Entrée) :

```
!pip install pm4py
```

3. À gauche, icône **Dossier** > **Importer** : importez les deux fichiers fournis.
   → *les fichiers restent disponibles pendant la session ; il faudra les réimporter à la prochaine ouverture.*

A.2 — Charger le journal

1. Nouvelle cellule :
   → *on indique à PM4Py quelle colonne représente le cas, l'activité et l'horodatage.*

```
import pandas as pd, statistics
import pm4py

df = pd.read_csv("demande_conge.csv")
df = pm4py.format_dataframe(df, case_id="case_id",
    activity_key="activity", timestamp_key="timestamp")
df.head(10)
```

A.3 — Statistiques de base

```
print("Nombre de cas :", df["case:concept:name"].nunique())
print("Nombre d'événements :", len(df))
print(df["concept:name"].value_counts())
```

1. Notez le nombre de cas, d'événements et d'activités. Repérez les activités qui ne figurent **pas** dans le processus prévu.

A.4 — Les variantes

```
variantes = pm4py.get_variants(df)
print("Nombre de variantes :", len(variantes))
for v, n in sorted(variantes.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(n, " -> ".join(v))
```

1. Une **variante** est un ordre d'activités suivi par au moins un cas. Comparez les deux variantes les plus fréquentes : en quoi diffèrent-elles ?
   → *elles ne diffèrent que par l'ordre d'« Enregistrer le congé » et de « Mettre à jour le planning » : ces deux activités sont faites en parallèle.*

Partie B : Découvrir le processus réel

B.1 — Le graphe des enchaînements

```
dfg, debuts, fins = pm4py.discover_dfg(df)
pm4py.view_dfg(dfg, debuts, fins)
```

1. Chaque flèche indique combien de fois une activité a été **directement suivie** par une autre. Repérez les flèches inattendues.
   → *ce graphe est simple à lire, mais il ne distingue pas un choix d'un parallélisme.*

B.2 — Un modèle BPMN découvert automatiquement

```
bpmn = pm4py.discover_bpmn_inductive(df, noise_threshold=0.2)
pm4py.view_bpmn(bpmn)
```

1. Observez le modèle obtenu avec l'**Inductive Miner** : retrouvez la passerelle parallèle, la boucle de refus et de modification, et la relance du manager.
   → *l'Inductive Miner garantit un modèle correct (sans blocage) ; le paramètre `noise_threshold` ignore les comportements très rares.*
2. Relancez avec `noise_threshold=0.0`, puis comparez les deux modèles.
   → *sans filtrage, tous les comportements, même rares, apparaissent : le modèle devient plus compliqué.*

Partie C : Vérifier la conformité

C.1 — Charger le modèle de référence

```
reference = pm4py.read_bpmn("modele_reference_conge.bpmn")
pm4py.view_bpmn(reference)
net, im, fm = pm4py.convert_to_petri_net(reference)
```

1. Le modèle est converti en **réseau de Petri**, le format utilisé par les algorithmes de conformité.

C.2 — Fitness et précision

```
print(pm4py.fitness_alignments(df, net, im, fm))
precision = pm4py.precision_alignments(df, net, im, fm)
print("Précision :", precision)
```

1. Relevez `log_fitness` (le journal est-il conforme au modèle ?), `percentage_of_fitting_traces` (pourcentage de cas parfaitement conformes) et la précision (le modèle autorise-t-il des comportements jamais observés ?).
   → *une fitness de 1 signifie que tous les cas suivent exactement le modèle.*

C.3 — Le type de chaque écart

```
from collections import Counter

alignements = pm4py.conformance_diagnostics_alignments(df, net, im, fm)
ecarts = Counter()
for a in alignements:
    for dans_journal, dans_modele in a["alignment"]:
        if dans_modele == ">>":
            ecarts["En trop : " + dans_journal] += 1
        elif dans_journal == ">>" and dans_modele is not None:
            ecarts["Manquant : " + dans_modele] += 1

for ecart, n in ecarts.most_common():
    print(n, ecart)
```

1. « **En trop** » : une activité réalisée mais non prévue par le modèle. « **Manquant** » : une activité prévue mais sautée.
   → *un « Manquant : Valider la demande » est une violation grave : un congé a été accordé sans l'accord du manager.*

C.4 — Les demandes enregistrées sans validation

```
sans_validation = pm4py.filter_event_attribute_values(
    df, "concept:name", ["Valider la demande"], level="case", retain=False)
nb = sans_validation["case:concept:name"].nunique()
print(nb, "demandes sans validation")
print(sans_validation["case:concept:name"].unique()[:10])
```

Partie D : Analyser la performance

D.1 — Durée des demandes

```
durees = pm4py.get_all_case_durations(df)          # en secondes
jours = [x / 86400 for x in durees]
print("Durée moyenne :", round(statistics.mean(jours), 2), "jours")
print("Durée médiane :", round(statistics.median(jours), 2), "jours")
print("Durée maximale :", round(max(jours), 1), "jours")
```

1. Comparez la durée moyenne à l'objectif de l'entreprise : **2 jours**.
   → *la médiane est souvent plus parlante que la moyenne : quelques cas très longs font monter la moyenne.*

D.2 — Le graphe de performance

```
perf, debuts, fins = pm4py.discover_performance_dfg(df)
pm4py.view_performance_dfg(perf, debuts, fins)
```

1. Sur chaque flèche s'affiche le temps moyen entre deux activités. Repérez les attentes les plus longues.
   → *une longue attente entre deux activités révèle un goulot d'étranglement.*

D.3 — Temps d'attente avant validation, par manager

```
d = df.sort_values(["case:concept:name", "time:timestamp"]).copy()
g = d.groupby("case:concept:name")
d["suivante"] = g["concept:name"].shift(-1)
d["t_suivante"] = g["time:timestamp"].shift(-1)
d["par"] = g["resource"].shift(-1)
v = d[d["suivante"] == "Valider la demande"].copy()
attente = v["t_suivante"] - v["time:timestamp"]
v["attente_h"] = attente.dt.total_seconds() / 3600
print(v.groupby("par")["attente_h"].agg(["count", "mean"]).round(1))
```

1. Quel manager fait attendre ses équipes beaucoup plus que les autres ?
   → *l'analyse par ressource transforme un constat global (« c'est lent ») en piste d'action précise.*

Questions (à rendre dans le compte rendu)

Q1. Donnez le nombre de cas, d'événements, d'activités et de variantes du journal. Quelles activités ne figurent pas dans le processus prévu ?

Q2. Quelle proportion des demandes suit l'une des deux variantes les plus fréquentes ? Pourquoi ces deux variantes représentent-elles en réalité le même chemin ?

Q3. Quelle est la fitness du journal par rapport au modèle de référence ? Quel pourcentage de demandes est parfaitement conforme ?

Q4. Classez les écarts trouvés en écarts **graves** et en écarts **acceptables**, en justifiant. Comment le moteur Camunda aurait-il empêché les validations sautées ?

Q5. L'objectif de 2 jours est-il respecté ? Quel est le principal goulot du processus ? Proposez une amélioration du modèle BPMN (par exemple avec une minuterie attachée).